

MODUL PRAKTIKUM V HISTOGRAM

Dosen Pengampu:

Mohamad Yasin, S.Kom, M.Kom



MODUL PRAKTIKUM HISTOGRAM CITRA

Untuk Memenuhi Tugas Mata Kuliah Pengolahan Citra Digital
Yang Diampu Oleh

Mohamad Yasin, S.Kom., M.Kom.



Disusun oleh:

Dana Rayka Mujahidin	230312607384
Nasywa Salsabila	230312605053
Shinta Arum Sari	230312606001

**UNIVERSITAS NEGERI MALANG
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
S1 MATEMATIKA
SEPTEMBER 2026**

Tujuan Praktikum:

1. Memahami konsep dasar histogram sebagai representasi grafis dari distribusi intensitas piksel pada citra digital.
2. Menjelaskan hubungan antara histogram dengan tingkat *exposure* (pencahayaan) dan kontras suatu citra.
3. Menguraikan serta menganalisis penerapan teknik pengolahan histogram, seperti perhitungan dan plotting histogram, contrast stretching, histogram equalization, dan histogram warna.
4. Mengimplementasikan praktikum histogram untuk analisis dan perbaikan kualitas citra digital menggunakan teknik pemrograman.

A. TEORI

Histogram merupakan representasi grafis dari distribusi data. Pada pengolahan citra digital, data yang ditampilkan pada histogram adalah persebaran frekuensi piksel yang memiliki nilai intensitas yang sama. Nilai intensitas pada histogram biasanya disebut “bin”. Untuk citra grayscale, ukuran bin di dalam histogram adalah 256 karena rentang intensitas berkisar 0–255. Histogram bersifat neighborhood-independent, artinya hanya menghitung sebaran nilai piksel tanpa memperhatikan posisi spasial atau hubungan antar piksel tetangganya.

Berikut adalah beberapa aspek mendasar yang tergolong dalam pengolahan histogram citra:

1. Histogram dan Exposure Citra

Histogram dapat digunakan untuk melakukan image enhancement atau menentukan nilai threshold. Histogram berguna untuk menganalisis *exposure*, *brightness*, *kontras*, dan *dynamic range* suatu citra. Suatu citra dapat dikatakan ter-expose dengan baik (*properly exposed*) ketika seluruh bin dari ujung kanan ke ujung kiri terisi tanpa ada space yang kosong. Citra yang *underexposed* memiliki histogram yang menumpuk di sisi kiri (gelap), sedangkan citra yang *overexposed* memiliki histogram yang menumpuk di sisi kanan (terang).

Low, Normal, High Contrast

Selain exposure, histogram juga dapat digunakan untuk menganalisis tingkat kontras suatu citra, yang terlihat dari lebar sebaran histogramnya. Citra dengan *low contrast* memiliki histogram yang mengumpul pada rentang nilai intensitas yang sempit, sehingga perbedaan antara area gelap dan terang kurang terlihat jelas. Citra dengan *normal contrast* memiliki sebaran histogram yang lebih merata pada rentang intensitas menengah. Sedangkan citra dengan *high contrast* memiliki histogram yang tersebar hingga menggunakan rentang bin yang lebih luas, sehingga perbedaan gelap-terang terlihat tajam. Pada praktikum ini, citra dengan *low contrast* dipilih sebagai studi kasus karena efek dari teknik perbaikan kontras (contrast stretching dan histogram equalization) akan terlihat paling signifikan pada jenis citra ini.

2. Perhitungan Histogram

Perhitungan histogram dapat dilakukan dengan dua cara, yaitu menggunakan OpenCV dengan fungsi `cv2.calcHist()`, maupun menggunakan NumPy dengan fungsi `np.histogram()`. Fungsi `cv2.calcHist()` menerima lima parameter utama, yaitu `images` (citra sumber), `channels` (channel yang dihitung histogramnya, [0] untuk grayscale), `mask` (None jika histogram dihitung dari keseluruhan citra), `histSize` (jumlah bin, umumnya [256]), dan `ranges` (rentang nilai, umumnya [0, 256]).

```
hist = cv2.calcHist([img], [0], None, [256], [0, 256])
# atau dengan NumPy:
hist, bins = np.histogram(img.ravel(), 256, [0, 256])
```

3. Plotting Histogram

Plotting histogram dapat dilakukan dengan berbagai cara, salah satunya menggunakan fungsi `matplotlib.pyplot.hist()` yang menghitung sekaligus menampilkan histogram secara langsung, maupun dengan `plt.plot()` untuk memplot hasil dari `cv2.calcHist()`.

4. Contrast Stretching / Histogram Stretching

Contrast stretching adalah teknik *image enhancement* yang bertujuan memperbaiki citra dengan kontras rendah (*low contrast*) agar batas antara area terang dan gelap menjadi lebih jelas, dengan cara meregangkan sebaran histogram sehingga mencakup rentang intensitas yang lebih lebar (0–255). Formula yang digunakan adalah sebagai berikut:

$$g(x, y) = \left[\frac{f(x, y) - f_{\min}}{f_{\max} - f_{\min}} \right] \times 255$$

Keterangan:

- $f(x, y)$: nilai piksel awal pada koordinat (x, y) .
- $f_{\min}$: nilai piksel terkecil (minimum) pada citra.
- $f_{\max}$: nilai piksel terbesar (maksimum) pada citra.
- $g(x, y)$: nilai piksel hasil stretching.

5. Histogram Equalization

Histogram equalization adalah suatu proses perataan histogram, di mana distribusi nilai derajat keabuan pada suatu citra dibuat merata menggunakan fungsi distribusi kumulatif (*cumulative distribution function/CDF*) dari histogram. Sifat-sifat histogram equalization antara lain: gray level yang sering muncul akan lebih dijarangkan jaraknya dengan gray level sebelumnya, gray level yang jarang muncul bisa lebih dirapatkan jaraknya, dan histogram baru pasti mencapai nilai maksimal keabuan (misal 255). Pada OpenCV, proses ini dapat dilakukan dengan fungsi:

```
dst = cv2.equalizeHist(src)
```

6. Histogram Warna

Komposisi warna pada citra berwarna (RGB) dapat direpresentasikan dalam bentuk histogram untuk tiap intensitas warna dalam citra. Untuk mendefinisikan histogram warna, tiap channel (biru, hijau, merah) dihitung dan diplot histogramnya secara terpisah, sehingga dapat diketahui distribusi dominasi warna pada suatu citra.

B. PERSIAPAN PRAKTIKUM

Sebelum melakukan praktikum, lakukan langkah-langkah berikut terlebih dahulu.

1. Instalasi dan Konfigurasi Python

Praktikum histogram dilakukan menggunakan bahasa pemrograman Python dengan bantuan Visual Studio Code sebagai code editor. Sebelum praktikum, pastikan Python telah terpasang dengan memeriksa melalui Terminal:

```
python --version
```

2. Instalasi Library

Praktikum ini membutuhkan library OpenCV, NumPy, dan Matplotlib. Instalasi dapat dilakukan via Terminal:

```
pip install opencv-python numpy matplotlib
```

3. Persiapan Folder Praktikum

Buat sebuah folder khusus untuk menyimpan gambar dan skrip Python. Contoh struktur folder:

```
Praktikum_Histogram: gambar.jpg, praktikum1.py, praktikum2.py, dst.
```

4. Persiapan Citra

Siapkan minimal satu file gambar (berwarna dan/atau grayscale) dengan kontras rendah (low contrast) agar efek dari contrast stretching maupun histogram equalization dapat terlihat jelas perbedaannya sebelum dan sesudah pemrosesan.

C. CONTOH PRAKTIKUM

Catatan: seluruh contoh berikut dijalankan menggunakan citra contoh (IMG_9549.jpg) yang sengaja dibuat dengan kontras rendah agar efek tiap teknik histogram terlihat jelas. Saat praktik/demo langsung, gambar ini dapat diganti dengan foto milik kelompok kalian sendiri caranya cukup mengganti nama file pada baris `cv2.imread()`.



Gambar 1. Citra Contoh (*Low Contrast*) yang Digunakan pada Seluruh Demo

1. Menampilkan Histogram Citra Grayscale

Operasi untuk menghitung dan menampilkan sebaran intensitas piksel dari sebuah citra grayscale menggunakan `cv2.calcHist()`. Digunakan untuk mengetahui karakteristik pencahayaan dan kontras suatu citra sebelum diproses lebih lanjut.

Kode Program:

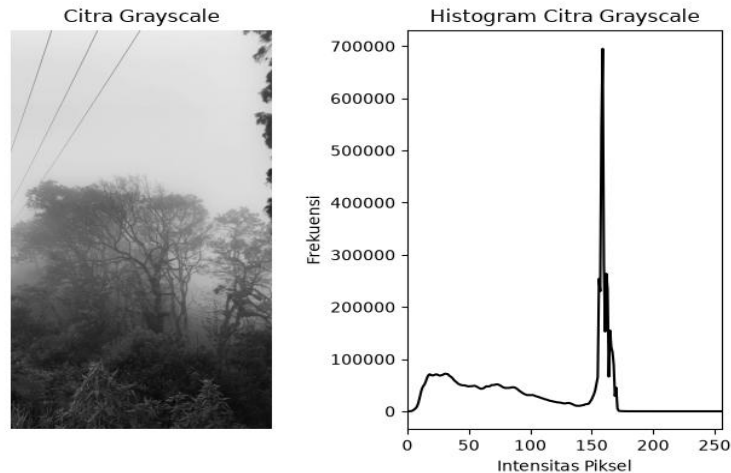
```
import cv2
import matplotlib.pyplot as plt

# 1. Baca citra dan ubah ke Grayscale
img = cv2.imread("IMG_9549.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# 2. Hitung histogram dengan OpenCV
hist = cv2.calcHist([gray], [0], None, [256], [0, 256])

# 3. Visualisasi
plt.subplot(1, 2, 1); plt.imshow(gray, cmap="gray")
plt.title("Citra Grayscale"); plt.axis("off")

plt.subplot(1, 2, 2); plt.plot(hist, color='k')
plt.title("Histogram Citra Grayscale")
plt.xlabel("Intensitas Piksel"); plt.ylabel("Frekuensi")
plt.xlim([0, 256])
plt.tight_layout(); plt.show()
```



Gambar 2. Output Histogram Citra Grayscale

Hasil Visual & Analisis: Histogram menunjukkan bahwa seluruh piksel citra hanya tersebar pada rentang intensitas yang sempit ($\approx 100\text{--}160$), dengan puncak tinggi yang mengumpul di satu area. Ini merupakan ciri khas citra dengan kontras rendah (*low contrast*), karena tidak ada piksel yang mengisi bin-bin di ujung kiri (gelap pekat) maupun ujung kanan (terang pekat).

2. Histogram Warna (RGB Channel)

Operasi untuk menghitung histogram dari citra berwarna secara terpisah pada tiap channel warna (Biru, Hijau, Merah). Digunakan untuk menganalisis komposisi dan dominasi warna pada suatu citra.

Kode Program:

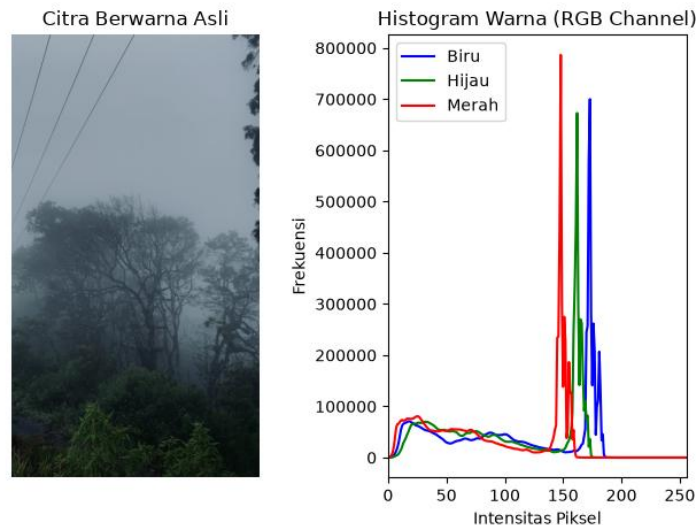
```
import cv2
import matplotlib.pyplot as plt

img = cv2.imread("IMG_9549.jpg") # ganti sesuai nama file gambar kalian

plt.subplot(1, 2, 1)
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.title("Citra Berwarna Asli"); plt.axis("off")

plt.subplot(1, 2, 2)
warna = ('b', 'g', 'r'); label = ('Biru', 'Hijau', 'Merah')
for i, col in enumerate(warna):
    hist = cv2.calcHist([img], [i], None, [256], [0, 256])
    plt.plot(hist, color=col, label=label[i])
plt.title("Histogram Warna (RGB Channel)")
plt.xlabel("Intensitas Piksel"); plt.ylabel("Frekuensi")
plt.xlim([0, 256]); plt.legend()
plt.tight_layout(); plt.show()
```

```
python --version
pip install opencv-python numpy matplotlib
Praktikum_Histogram: gambar.jpg, praktikum1.py, praktikum2.py, dst.
```



Gambar 3. Output Histogram Warna

Hasil Visual & Analisis: Ketiga channel warna (biru, hijau, merah) memiliki puncak histogram yang saling berdekatan namun tidak persis sama, menunjukkan adanya sedikit dominasi warna tertentu (dalam contoh ini, citra cenderung kebiruan/keabuan pada bagian langit). Pola ini konsisten dengan tampilan visual citra yang didominasi warna abu-abu kebiruan.

3. Contrast Stretching

Operasi peregangan histogram untuk memperbaiki citra dengan kontras rendah, dengan cara memetakan ulang nilai intensitas minimum dan maksimum citra ke rentang penuh 0–255. Digunakan untuk mempertajam batas antara area terang dan gelap pada citra.

Kode Program:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread("IMG_9549.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

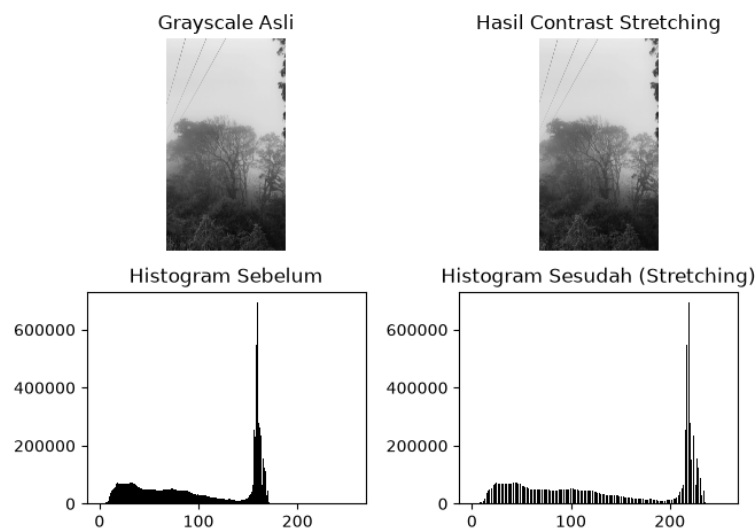
# Contrast Stretching manual:
# g(x,y) = (f(x,y) - fmin) / (fmax - fmin) * 255
fmin, fmax = gray.min(), gray.max()
stretched = (gray.astype(np.float32) - fmin) / (fmax - fmin) * 255
```

```

stretched = np.clip(stretched, 0, 255).astype(np.uint8)

plt.subplot(2, 2, 1); plt.imshow(gray, cmap="gray")
plt.title("Grayscale Asli"); plt.axis("off")
plt.subplot(2, 2, 2); plt.imshow(stretched, cmap="gray")
plt.title("Hasil Contrast Stretching"); plt.axis("off")
plt.subplot(2, 2, 3); plt.hist(gray.ravel(), 256, (0, 256), color='k')
plt.title("Histogram Sebelum")
plt.subplot(2, 2, 4); plt.hist(stretched.ravel(), 256, (0, 256), color='k')
plt.title("Histogram Sesudah (Stretching)")
plt.tight_layout(); plt.show()

```



Gambar 4. Output *Contrast Stretching*

Hasil Visual & Analisis: Sebelum diproses, nilai piksel hanya berkisar antara 101–152. Setelah contrast stretching, rentang nilai piksel berhasil diregangkan menjadi 0–255 secara penuh. Secara visual, citra hasil terlihat jauh lebih tajam dan detail antar objek (gunung, pohon, matahari) menjadi lebih mudah dibedakan dibandingkan citra aslinya.

4. Histogram Equalization

Operasi perataan histogram menggunakan fungsi distribusi kumulatif untuk menyebarkan nilai intensitas piksel secara lebih merata ke seluruh rentang keabuan. Digunakan untuk meningkatkan kontras citra secara otomatis, khususnya pada citra dengan histogram yang menumpuk di satu area sempit.

Kode Program:

```

import cv2
import matplotlib.pyplot as plt

```

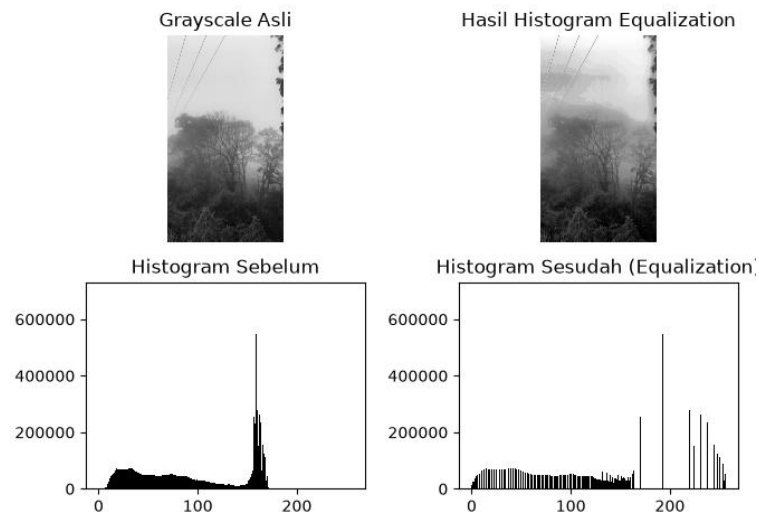
```

img = cv2.imread("IMG_9549.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

equ = cv2.equalizeHist(gray)

plt.subplot(2, 2, 1); plt.imshow(gray, cmap="gray")
plt.title("Grayscale Asli"); plt.axis("off")
plt.subplot(2, 2, 2); plt.imshow(equ, cmap="gray")
plt.title("Hasil Histogram Equalization"); plt.axis("off")
plt.subplot(2, 2, 3); plt.hist(gray.ravel(), 256, (0, 256), color='k')
plt.title("Histogram Sebelum")
plt.subplot(2, 2, 4); plt.hist(equ.ravel(), 256, (0, 256), color='k')
plt.title("Histogram Sesudah (Equalization)")
plt.tight_layout(); plt.show()

```



Gambar 5. Output Histogram Equalization

Hasil Visual & Analisis: Histogram equalization berhasil menyebarkan nilai piksel ke rentang yang jauh lebih lebar, membuat kontras citra meningkat drastis dibanding citra asli. Namun, terlihat pula efek samping berupa noise/grain yang lebih terasa pada area yang tadinya nyaris seragam (misalnya langit), karena proses equalization ikut meregangkan perbedaan intensitas yang sangat kecil akibat noise menjadi lebih terlihat.

D. LATIHAN SOAL

Buatlah sebuah program Python menggunakan OpenCV dan Matplotlib untuk mengolah file gambar.png dengan ketentuan sebagai berikut: baca citra berwarna, lalu tampilkan histogram warna (RGB channel) dari citra tersebut. Selanjutnya, konversi citra menjadi grayscale dan

terapkan contrast stretching pada citra grayscale tersebut. Tampilkan hasilnya dalam satu jendela plot yang memuat: Citra Berwarna Asli, Histogram Warna (RGB), Citra Grayscale Asli, dan Hasil Contrast Stretching.

Jawaban

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# 1. Membaca citra berwarna
img = cv2.imread("stiker wa.jpg")

# 2. Konversi ke Grayscale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# 3. Contrast Stretching pada citra grayscale
fmin, fmax = gray.min(), gray.max()
stretched = (gray.astype(np.float32) - fmin) / (fmax - fmin) * 255
stretched = np.clip(stretched, 0, 255).astype(np.uint8)

# 4. Visualisasi Hasil Latihan
plt.figure(figsize=(10, 8))
plt.subplot(2, 2, 1)
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.title("1. Citra Berwarna Asli"); plt.axis("off")

plt.subplot(2, 2, 2)
warna = ('b', 'g', 'r')
for i, col in enumerate(warna):
    hist = cv2.calcHist([img], [i], None, [256], [0, 256])
    plt.plot(hist, color=col)
plt.title("2. Histogram Warna (RGB)")

plt.subplot(2, 2, 3)
plt.imshow(gray, cmap="gray")
plt.title("3. Grayscale Asli"); plt.axis("off")

plt.subplot(2, 2, 4)
plt.imshow(stretched, cmap="gray")
plt.title("4. Hasil Contrast Stretching"); plt.axis("off")

plt.tight_layout()
plt.show()
```

E. SOAL

Buatlah skrip Python untuk mengolah file gambar.png menjadi citra grayscale, lalu terapkan Histogram Equalization menggunakan `cv2.equalizeHist()`. Tampilkan keempat hasil berikut secara berdampingan dalam grid 2×2 menggunakan Matplotlib, lengkap dengan judul sub-plot: Citra Grayscale Asli, Histogram Grayscale Asli, Citra Hasil Equalization, dan Histogram Hasil Equalization. Pada laporan, berikan analisis singkat mengenai mengapa histogram hasil equalization dapat terlihat “terputus-putus” (bergerigi/tidak kontinu) dibandingkan dengan histogram citra aslinya!